

C Tokens

The building blocks of C programming

Understanding tokens is fundamental in C programming as they form the building blocks of C code. Tokens represent the smallest individual units of a C program, aiding in syntax analysis and code interpretation.

Definition and Types of Tokens



Definition

Tokens in C programming are the smallest individual units constituting a C program. They include keywords, identifiers, constants, strings, operators, and special symbols.



Types of Tokens

C tokens are categorized into:

- Keywords (e.g., `int`, `if`, `while`)
- Identifiers (user-defined names)
- Constants (numeric or character literals)
- Strings (sequences of characters enclosed in double quotes)
- Operators (arithmetic, relational, logical)
- Special symbols (punctuation characters)

Keywords and Identifiers



Keywords

Keywords are reserved words with predefined meanings in C. Examples include ``if``, ``else``, ``for``, ``int``, and ``void``. They cannot be used as identifiers.

Common C Keywords:

`int`, `float`, `double`, `char`, `if`, `else`, `for`, `while`, `do`,
`switch`, `case`, `break`, `continue`, `return`, `void`, `struct`,
`union`, `enum`, `sizeof`, `typedef`, `const`, `static`, `extern`,
`register`, `auto`, `volatile`, `goto`, `default`, `signed`,
`unsigned`, `short`, `long`



Identifiers

Identifiers are user-defined names used to represent variables, functions, or other entities in a C program. They consist of letters, digits, and underscores, beginning with a letter or underscore.

Valid Identifiers:

`count`, `_total`, `sum1`, `calculate_average`, `MAX_VALUE`

Invalid Identifiers:

2count (starts with digit), total-value (contains hyphen), int (keyword)

3

Constants



Numeric Constants

Numeric constants represent fixed numerical values, such as integers (`123`, `-45`), floating-point numbers (`3.14`, `0.75`), and scientific notation (`2.5e3`).

Examples:

```
int age = 25; // Integer constant
```

```
float pi = 3.14; // Floating-point constant
```

```
double distance = 2.5e3; // Scientific notation
```

```
long number = 123456789L; // Long integer constant
```



Character Constants

Character constants are single characters enclosed in single quotes ('A', '5', '%'), representing their ASCII or Unicode values.

Examples:

```
char grade = 'A'; // Character constant
```

```
char digit = '5'; // Digit character
```

```
char symbol = '%'; // Symbol character
```

```
char newline = '\n'; // Escape sequence
```

4

Strings and Escape Sequences



String Constants

Strings are sequences of characters enclosed in double quotes ("Hello, World!"). They can contain alphanumeric characters, symbols, and escape sequences ('\n', '\t', '\"') for special characters.

Examples:

```
char greeting[] = "Hello, World!";
```

```
char message[] = "Line 1\nLine 2"; // With newline
```

```
char path[] = "C:\\Program Files"; // With backslash
```

```
char quote[] = "She said, \"Hi!\""; // With quotes
```



Escape Sequences

Escape sequences are special combinations of characters that represent non-printable or special characters within strings.

Common Escape Sequences:

`\n` - Newline

`\t` - Horizontal tab

`\r` - Carriage return

`\\` - Backslash

`\"` - Double quote

`\'` - Single quote

`\0` - Null character

Arithmetic Operators +

Arithmetic operators such as `+`, `-`, `*`, `/`, `%` perform arithmetic operations on numeric values.

Operator	Purpose	Example
+	Addition	<code>int sum = a + b;</code>
-	Subtraction	<code>int diff = a - b;</code>
*	Multiplication	<code>int product = a * b;</code>
/	Division	<code>int quotient = a / b;</code>
%	Remainder after integer division	<code>int remainder = a % b;</code>

```
// Example of arithmetic operators
int a = 10, b = 3;
int sum = a + b; // 13
int diff = a - b; // 7
int product = a * b; // 30
int quotient = a / b; // 3
int remainder = a % b; // 1
```

Relational and Logical Operators

Relational and logical operators ``==`, `!=`, `>`, `<`, `&&`, `||`` perform comparisons and logical operations.

Operator	Meaning	Example
==	Equality	<code>if (a == b)</code>
!=	Not equal to	<code>if (a != b)</code>
<	Less than	<code>if (a < b)</code>
>	Greater than	<code>if (a > b)</code>
<=	Less than or equal to	<code>if (a <= b)</code>
>=	Greater than or equal to	<code>if (a >= b)</code>
&&	Logical AND	<code>if (a > 0 && b > 0)</code>
 	Logical OR	<code>if (a > 0 b > 0)</code>
!	Logical NOT	<code>if (!(a > 0))</code>

Assignment and Increment/Decrement Operators

Assignment and increment/decrement operators `=`, `+=`, `-=`, `++`, `--` modify variable values.

=

Assignment

Assigns a value to a variable

+=

Addition Assignment

Adds and assigns

-=

Subtraction Assignment

Subtracts and assigns

***=**

Multiplication Assignment

Multiplies and assigns

/=

Division Assignment

Divides and assigns

%=

Modulus Assignment

Calculates remainder and assigns

++

Increment

Increases value by 1

--

Decrement

Decreases value by 1

```
// Example of assignment and increment/decrement operators
int a = 10;
a += 5; // a is now 15
a -= 3; // a is now 12
a *= 2; // a is now 24
a /= 4; // a is now 6
a++; // a is now 7
a--; // a is now 6
```

Special Symbols ✨

Special symbols include punctuation characters like parentheses `()`, braces `{}`, brackets `[]`, commas `,`, semicolons `;`, and colons `:` used to structure C code and define blocks, expressions, and statements.

()

Parentheses

Function calls,
expressions

{ }

Braces

Code blocks, function
bodies

[]

Brackets

Array indexing

,

Comma

Separates list items

;

Semicolon

Statement terminator

:

Colon

Labels, conditional
operator

.

Dot

Structure member
access

→

Arrow

Pointer to structure
member

```
// Example of special symbols in C
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
}
```

```
return 0;  
}
```

9

Tokenization and Lexical Analysis



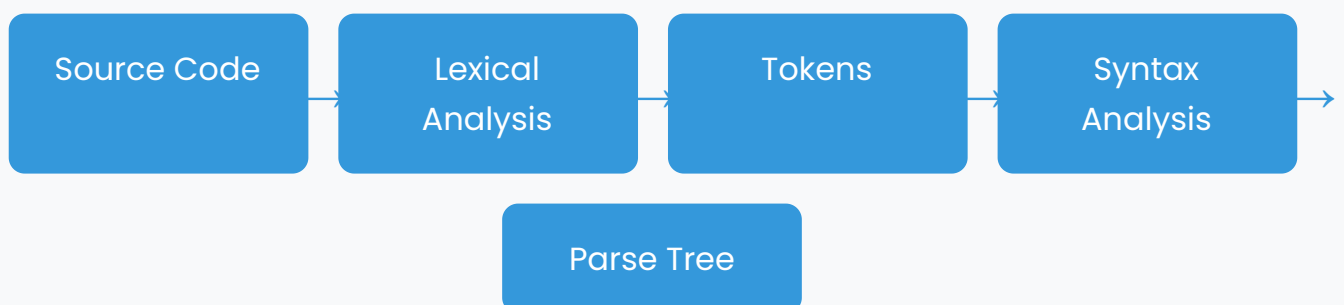
Lexical Analysis

The process of breaking down a C program into tokens is called lexical analysis. Tokenization involves identifying tokens and categorizing them based on their types.



Tokenization Tools

Lexical analyzers and tokenizers are used to scan source code, recognize tokens, and pass them to the parser for further syntactic analysis.



```
// Example of tokenization
// Source code: int sum = a + b;

// Tokens:
// 1. Keyword: int
// 2. Identifier: sum
// 3. Operator: =
// 4. Identifier: a
// 5. Operator: +
// 6. Identifier: b
// 7. Special symbol: ;
```

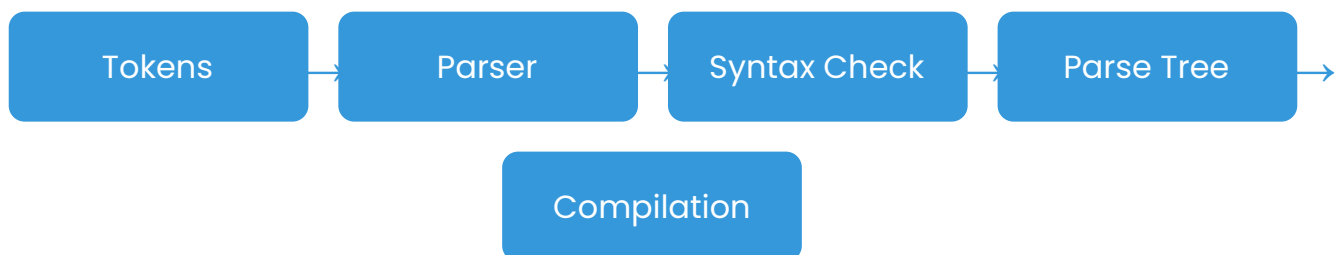
10

Role in Syntax Analysis



Syntax Parsing

Tokens serve as the input for the parser during syntax analysis. The parser uses tokens to analyze the structure and syntax of the program based on predefined grammar rules.



Importance of Tokens

- Tokens are the fundamental building blocks of C programs
- They enable the compiler to understand the structure of the code

- Proper tokenization is essential for correct syntax analysis
- Errors in tokenization can lead to compilation failures

Understanding tokens is crucial for both writing correct C code and for comprehending how the C compiler processes your programs. By recognizing the different types of tokens and their roles, you can write more efficient and error-free code.